

AWS Load Balancer Practical

2 Week – Day 7 Class

Topic

Create Two NGINX Application Servers using AMI and Configure Application Load Balancer (ALB)

Architecture Flow

Browser



Application Load Balancer (ALB)



Target Group



Server 1

Server 2

(NGINX)

(NGINX)



AWS Classes

DevOps Classes

Important Note

Linux Server Connectivity

In this class:

- Linux EC2 server connectivity is done through AWS Console browser terminal only.

Navigation:

- [AWS Console](#)
- EC2
- Select Instance
- Connect
- EC2 Instance Connect
- Connect

Linux Connectivity Flow

AWS Console



EC2 Instance Connect



Linux Ubuntu Server

Lab Objective

In this class we will:

1. Create Application Server 1
 2. Install NGINX
 3. Create Custom Index Page
 4. Create AMI from Server 1
 5. Launch Server 2 using AMI
 6. Modify Server 2 Index Page
 7. Verify Both Servers
 8. Create Target Group
 9. Create Application Load Balancer
 10. Attach Both Servers
 11. Access ALB URL
 12. Verify Load Balancing
-

Step 1 – Launch Ubuntu EC2 Server 1

Open: [AWS Console](#)

Go to:

- Services

- EC2
- Launch Instance

Configure Server 1

Setting	Value
Name	App-Server-1
AMI	Ubuntu Server
Instance Type	t2.micro
Security Group	Allow 22,80
Public IP	Enable

Launch Server.

Step 2 – Connect to Server 1

Go to:

- EC2
 - Instances
 - Select App-Server-1
 - Click Connect
 - EC2 Instance Connect
 - Connect
-

Step 3 – Update Packages

Run:

```
sudo apt update -y
```

Step 4 – Install NGINX

Run:

```
sudo apt install nginx -y
```

Step 5 – Start and Enable NGINX

Run:

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

Step 6 – Create Application Page in Server 1

Go to website directory:

```
cd /var/www/html
```

Edit index page:

```
sudo vi index.html
```

Press:

i

Paste:

```
<html>
```

```
<head>
```

```
<title>AWS Classes</title>
```

```
</head>
```

```
<body style="background-color:black; color:white; text-align:center; padding-top:200px;">
```

```
<h1>WELCOME AWS Classes</h1>
```

```
<h2>Application Server 1</h2>
```

```
</body>
```

</html>

Save:

ESC → :wq

Step 7 – Verify Server 1

Open Browser:

http://SERVER1-PUBLIC-IP

Expected Output:

WELCOME AWS Classes

Application Server 1

Step 8 – Create AMI from Server 1

Go to:

- EC2 Console
- Instances
- Select Server 1

Click:

- Actions
 - Image and Templates
 - Create Image
-

Configure AMI

Setting	Value
---------	-------

AMI Name	nginx-app-ami
----------	---------------

Click:

- Create Image

Wait until:

- AMI Status = Available

AMI Flow

Application Server 1



Create AMI



Reusable Server Image

Step 9 – Launch Server 2 using AMI

Go to:

- EC2
- AMIs

Select:

- nginx-app-ami

Click:

- Launch Instance

Configure Server 2

Setting	Value
Name	App-Server-2
Instance Type	t2.micro
Security Group	Allow 22,80
Public IP	Enable

Launch Instance.

Step 10 – Connect to Server 2

Go to:

- EC2
 - Instances
 - Select App-Server-2
 - Connect
 - EC2 Instance Connect
 - Connect
-

Step 11 – Modify Index Page in Server 2

Run:

```
cd /var/www/html
```

Edit:

```
sudo vi index.html
```

Press:

i

Paste:

```
<html>
```

```
<head>
```

```
<title>DevOps Classes</title>
```

```
</head>
```

```
<body style="background-color:blue; color:white; text-align:center; padding-top:200px;">
```

```
<h1>WELCOME DevOps Classes</h1>
```

```
<h2>Application Server 2</h2>
```

```
</body>
```

```
</html>
```

Save:

ESC → :wq

Step 12 – Verify Server 2

Open:

http://SERVER2-PUBLIC-IP

Expected:

WELCOME DevOps Classes

Application Server 2

Step 13 – Create Target Group

Go to:

- EC2 Console
 - Target Groups
 - Create Target Group
-

Configure Target Group

Setting	Value
---------	-------

Target Type	Instances
-------------	-----------

Name	nginx-target-group
------	--------------------

Protocol	HTTP
----------	------

Setting	Value
---------	-------

Port	80
------	----

Click:

- Next
-

Step 14 – Register Targets

Select:

- App-Server-1
- App-Server-2

Click:

- Include as Pending Below

Click:

- Create Target Group
-

Step 15 – Create Application Load Balancer

Go to:

- EC2 Console
- Load Balancers
- Create Load Balancer

Select:

- Application Load Balancer
-

Configure ALB

Setting	Value
---------	-------

Name	nginx-alb
------	-----------

Setting Value

Scheme Internet Facing

IP Type IPv4

Network Mapping

Select:

- Default VPC
 - Minimum 2 Availability Zones
-

Security Group

Allow:

- HTTP Port 80
-

Listener and Routing

Protocol Port

HTTP 80

Select Target Group:

- nginx-target-group

Click:

- Create Load Balancer
-

ALB Architecture

Browser



Application Load Balancer



Target Group



App Server 1

App Server 2

Step 16 – Verify Target Health

Go to:

- Target Groups
- Targets

Expected Status:

healthy

healthy

Step 17 – Access ALB URL

Copy:

- Load Balancer DNS Name

Open Browser:

http://LOAD-BALANCER-DNS

Expected Output

First Refresh

WELCOME AWS Classes

Application Server 1

Second Refresh

WELCOME DevOps Classes

Application Server 2

Final Architecture

Browser



Application Load Balancer



Target Group





Application Server 1



Application Server 2

WELCOME AWS Classes

WELCOME DevOps Classes

Real-Time Interview Questions

Q1. What is AMI?

Answer:

- Amazon Web Services AMI is a reusable image used to launch EC2 servers.

Q2. Why did you create AMI from Server 1?

Answer:

- To clone the server with same application and configurations.

Q3. What is Target Group?

Answer:

- Target Group forwards traffic from ALB to backend application servers.

Q4. What is Application Load Balancer?

Answer:

- Amazon Web Services ALB distributes incoming HTTP/HTTPS traffic across multiple servers.

Q5. Which layer does ALB work on?

Answer:

- Layer 7 (Application Layer).

Q6. How does ALB distribute traffic?

Answer:

- Using Round Robin method by default.

Q7. What happens if one server becomes unhealthy?

Answer:

- ALB sends traffic only to healthy servers.

Q8. Why did you use NGINX?

Answer:

- NGINX NGINX is lightweight and high-performance web server software.

Q9. How do you verify target health?

Answer:

- Target Groups → Targets → Health Status.

Q10. Explain complete project flow.

Browser



ALB DNS URL



Application Load Balancer



Target Group



Application Server 1 / Application Server 2



NGINX



Website Response

Sai Cloud Services